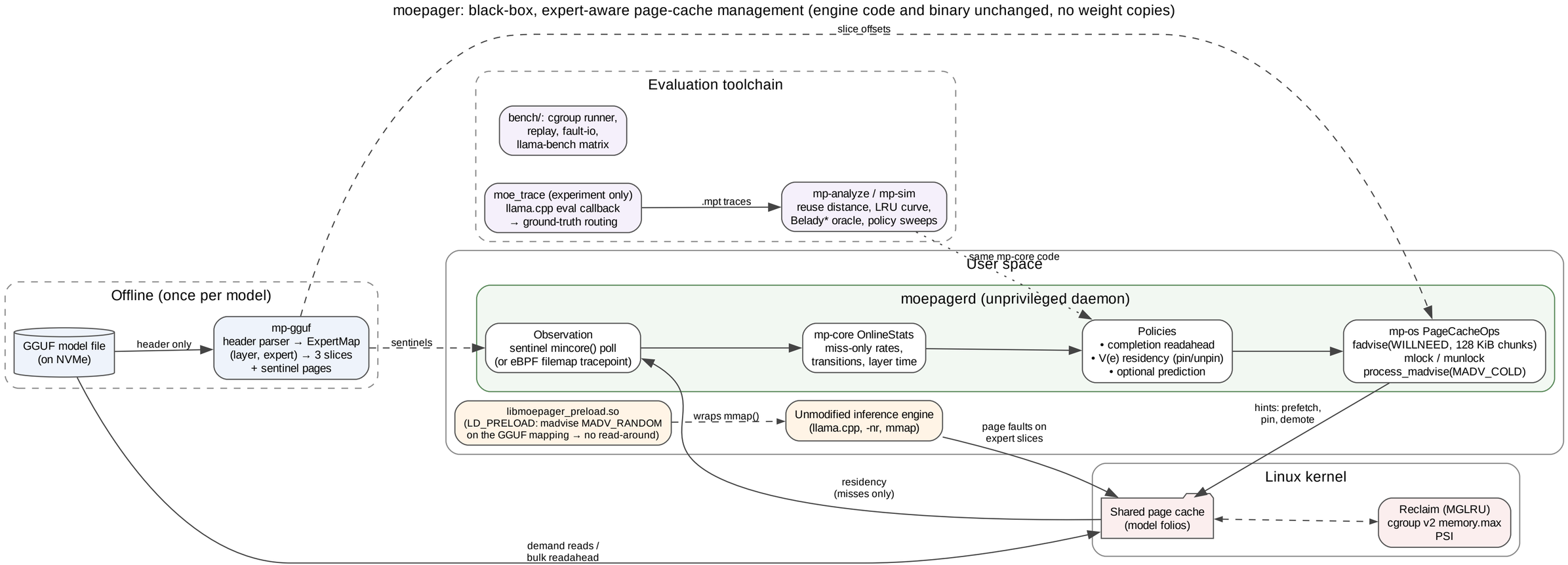
moepager: black-box, expert-aware page-cache management (engine code and binary unchanged, no weight copies)
slice offsets
Evaluation toolchain
bench/: cgroup runner, replay, fault-io, llama-bench matrix
moe_trace (experiment only) llama.cpp eval callback → ground-truth routing
.mpt traces
mp-analyze / mp-sim reuse distance, LRU curve, Belady* oracle, policy sweeps
same mp-core code
User space
Offline (once per model)
GGUF model file (on NVMe)
header only
mp-gguf header parser → ExpertMap (layer, expert) → 3 slices + sentinel pages
sentinels
moepagerd (unprivileged daemon)
Observation sentinel mincore() poll (or eBPF filemap tracepoint)
mp-core OnlineStats miss-only rates, transitions, layer time
Policies • completion readahead • V(e) residency (pin/unpin) • optional prediction
mp-os PageCacheOps fadvise(WILLNEED, 128 KiB chunks) mlock / munlock process_madvise(MADV_COLD)
libmoepager_preload.so (LD_PRELOAD: madvise MADV_RANDOM on the GGUF mapping → no read-around)
wraps mmap()
Unmodified inference engine (llama.cpp, -nr, mmap)
page faults on expert slices
hints: prefetch, pin, demote
residency (misses only)
Linux kernel
Shared page cache (model folios)
Reclaim (MGLRU) cgroup v2 memory.max PSI
demand reads / bulk readahead